

Design write-up

I went deep on **Layer 1 (Retrieval & Grounding)** and **Layer 2 (Tool Use & Agentic)**, plus the **Context Engineering** stretch. Those three compose: layers 1 and 2 decide what the model sees each turn, and the stretch is the budget imposed on top of them. Picking 1 and 3 instead would have left the stretch layer with nothing to budget.

There are eval artifacts (8 cases, a rubric, a scoring script) because the brief asks for them, but I did not treat Layer 3 as one of my two, and the write-up says so plainly at the end.

The idea holding it together is that the assistant checks its own answer before you read it. The criterion it cites is verified against the spec text, and the patch it proposes is run through a real browser.

Layer 1 — Retrieval & Grounding

The riskiest thing this assistant can do is name the wrong success criterion in confident prose. A developer who reads "SC 1.4.3 requires..." has no cheap way to check, and a wrong criterion sends them to the wrong fix. So criterion identity is never generated. axe-core already tags every rule it fires with the criterion behind it (`wcag111` maps to SC 1.1.1). That makes "which criterion applies" a dictionary lookup against the W3C's own `wcag.json`, which I vendored into the repo. No embeddings, no similarity threshold, nothing that can be confidently wrong. That is my answer to the brief's question about when the model's built-in knowledge is good enough: for criterion identity, never.

Chunks carry a flavour — normative, understanding, technique, or community — and the flavour decides how the chunk is presented, not just whether it gets retrieved. Normative text is quoted verbatim and labelled as the requirement. Techniques are labelled informative. The retrieval difference between them is small. The presentation difference is what stops the model writing "WCAG requires `aria-label`", which it does not; it requires an accessible name.

A second, deliberately narrow retrieval path handles follow-ups phrased as prose rather than rule IDs ("what if I use `aria-label` instead?"). It is BM25 and bge-small fused with reciprocal rank fusion. It never decides which criterion applies.

Citations get checked in three stages, cheapest first. Extract every `[SC 1.4.3]` and `[H37]` from the answer. Check each one was actually in the context supplied; if it wasn't, the model produced it from memory, which counts as fabrication even when the criterion is real and relevant. Then a small model decides whether the quoted spec text supports the claim, seeing only that text and the one sentence making the claim. The isolation matters: a judge shown everything the generator saw tends to inherit its blind spots. Bad citations render struck-through next to the real text rather than being deleted quietly, because silently laundering the answer makes it look more trustworthy while making it less so.

LCP has no normative anchor at all. The anchor path returns nothing and the prompt says so out loud, so the line between a conformance failure and a performance finding falls out of the architecture instead of being hand-written.

Not built: reranking, query rewriting, or a vector database. The retrieval that carries real risk is already exact, and 1,075 chunks is a 1.6 MB matrix and a dot product. Each of those three would add a failure mode to the one path that currently cannot fail.

Layer 2 — Tool Use & Agentic

The whole latency answer is that the first turn and later turns are not the same thing.

Turn 0 offers no tools. The context recipe has already pre-fetched everything a known issue needs, so "help me fix this" costs exactly one round trip and starts streaming immediately. From turn 1 the tools are unlocked, because a follow-up is unpredictable and you cannot pre-fetch for it.

Every tool description states its own cost, so the model can decide whether a question justifies the wait: index reads are instant, `contrast_ratio` is fast local maths, `validate_fix` launches a browser and takes 2–5 seconds. A model that cannot see cost cannot trade off against it.

`contrast_ratio` exists so the model never does that arithmetic itself. A wrong ratio is the perfect wrong-but-plausible failure. It looks like a fact, it is stated confidently, and nobody re-checks it.

`validate_fix` applies the model's proposed markup to the real page and re-runs axe against it. Not a detached fragment, because CSS decides contrast and the ancestor chain decides ARIA. It returns `cleared`, `partial` or `regressed` along with anything newly introduced, which is the brief's "fixed one thing, broke another" measured rather than assumed.

Tools never raise. They return one shape: a status of `ok`, `not_found`, `stale`, `ambiguous` or `timeout`, plus data, a reason and a suggestion. `not_found` hands back the three nearest selectors so a miss is a lead rather than an invitation to invent one. `stale` fires when the page hash has moved since the scan, and the tool then refuses to answer at all. Answering confidently from a stale index is worse than admitting the page changed.

A full re-scan is a button, not a tool. It costs 30 seconds of someone's time and the model should not spend that on its own initiative.

Nothing on the turn-0 path is allowed to block. Loading the dense encoder takes about 22 seconds on CPU, and that cost was landing on whoever clicked "explain and fix" first after a restart. It now loads on a background thread, and a turn arriving before it is ready runs BM25-only with the inspector saying so. This is only safe because the anchor path never used embeddings, so a cold encoder cannot change which criterion gets cited.

What the measurements changed

My first two guesses were both wrong, which is the useful part.

Tool latency was never the constraint. All the tool calls in a turn together came to about 50 ms against a multi-second turn, so parallelising them would have optimised 0.05% of it.

Context size was not the constraint either. Cutting the prompt from 4,462 to 1,854 tokens made time-to-first-token *worse*, 11.5 s against 9.6 s. What actually moves TTFT is cache state: the same prompt costs about 9.6 s on the write and 1.9 s on the read. Caching earns its place on cost, not on latency; measured against a guaranteed-cold cache, identical prompts with and without `cache_control` differed by less than run-to-run noise.

The lever was the model, and the eval suite settled it:

	eval mean	TTFT	full answer	output tokens
claude-opus-5	1.62	6,353 ms	15.8 s	935
claude-sonnet-5	1.51	1,252 ms	5.5 s	412
claude-haiku-4-5	1.51	797 ms	4.6 s	285

A 0.11 spread cannot show Opus is better here, so the default is Haiku: 797 ms to first token, 4.6 s complete. That result is the rest of the design arguing for itself. Once the criterion is looked up exactly, the contrast maths is computed rather than generated, and the page evidence is selected by recipe, what is left is explanation, and a small model explains well.

The exception is instructive. The one place smaller models wobbled was `lcp`, the only issue type with no normative anchor. Running Haiku's own patch through `validate_fix` three times gave cleared, regressed (four new violations, one critical), cleared. Weakest grounding, largest model dependence, exactly what this architecture predicts. So model choice is routed by grounding strength rather than set globally: anchored issue types get Haiku, the unanchored LCP path gets Sonnet.

Not built: parallel tool calls (they would optimise nothing), a tool that edits the developer's actual files (proposing a patch and applying one are different trust levels, and nothing here has earned the second), multi-page crawl, auth, accounts.

Stretch — Context Engineering

The uncomfortable part first: the DOM fits. Frontier context windows run to a million tokens and a government homepage is a few hundred thousand at worst. Capacity is not the constraint.

What cutting buys is cost, time-to-first-token, and mainly quality. Hand a model the whole DOM and it writes generic advice, because nothing in the payload tells it which 40 nodes out of 4,000 are the answer. Selection is the signal, and it is what let the assistant drop to a small model without the eval suite noticing.

Each recipe answers one question: which facts fully determine *this* fix?

Rule family	Keeps	Why
naming (<code>image-alt</code> , <code>link-name</code> , <code>label</code>)	element, landmark path, enclosing <code><a></code> / <code><figure></code> / <code><figcaption></code> , sibling text, filename, nearest heading	the real question is what the name should <i>say</i> , which lives in the prose, not the DOM shape
color-contrast	element, computed colour, the ancestor that actually paints the background, font size and weight	those five facts fix both the ratio and the threshold
ARIA	element, explicit and implicit role, parent and child roles	a role-graph problem, where surrounding text is noise
lcp	LCP element, its request, render-blocking <code><head></code> resources in document order	LCP is a chain, and the blocker is usually up in <code><head></code>

The contrast recipe is the argument for all of this. On the fixture the failing `<p>` has `background-color: rgba(0,0,0,0)` , so it is transparent, and the colour a user actually sees is painted by a `<section>` three levels up. Walking up gives 3.08:1. Assuming a white page background, which is what a model without that context does, gives 3.45:1 and a recommended colour that still fails. That one number is the whole case for issue-type-specific context.

Some smaller decisions that follow from the same thinking. Structure is serialised as compressed paths (`main > section[aria-label="News"] > figure` , about 15 tokens where full nodes cost 600). Styles are flat key-values, limited to the properties the rule depends on. Cache breakpoints are placed by stability rather than by section: system and tools frozen for the session, spec and page evidence frozen per issue, conversation volatile and uncached, so a ten-turn follow-up pays for the page context once. The index is keyed by `(url, content_hash)` , so many developers on one page share one index and one cached prefix, and staleness comes from re-hashing rather than a TTL. Dropped blocks keep their reason and still render in the inspector, because what got cut tells you more than what survived.

Answer length turned out to be part of the context budget too. Measured turns were spending about 100 seconds generating 3,000+ output tokens while every tool call in the same turn cost under 25 ms. Capping output and asking the prompt for the same brevity cut a representative case from 61 s to 23 s.

Not built: conversation summarising (I drop oldest turns instead, which is cruder but visible, and cannot silently lose a constraint stated three turns ago); recipes beyond the four families, so the rest get a labelled generic fallback and the UI says so; more than one performance issue type.

Evaluation, and what I would do next

Layer 3 was not one of my two, but the deliverable needs artifacts, so there are 8 cases and 5 dimensions. Three of the five are decided by running code: criterion correctness from the citation verifier, fix validity from putting the model's own patch through `validate_fix`, and collateral damage from what that same run newly introduced. A judge grades only the two that genuinely require reading.

That split is my answer to wrong-and-obviously-wrong versus wrong-but-plausible. The plausible ones worry me, so the dimensions where a plausible error does the most harm are not left to a judge that shares the generator's blind spots.

The suite is noisier than I originally claimed, and `evals/rubric.md` now says so. Three consecutive runs with no code changed scored 1.55, 1.30 and 1.46. I had documented a ± 0.10 band; it is closer to ± 0.25 . I was also wrong to call criterion correctness deterministic: its first two stages are pure code, but the support check is a model call. Fix validity and collateral damage are deterministic scorers, but over a non-deterministic input, since whether an answer contains an HTML block at all varies between runs. The honest claim is not that these dimensions are stable, it is that they cannot be fooled by fluent prose. That is still why they carry the weight.

Built deeply: the anchor path and citation verification, the four recipes and the budget, the tool failure contract, and the Context Inspector.

Left out: Layer 3 proper. No golden answers, no judge calibration, no regression suite across prompt versions.

Next, in this order. First, recipes for the top 20 axe rules by frequency, because coverage is what makes this usable and the fallback produces exactly the generic advice this design argues against. Second, more eval cases and repeated runs per case, because a ± 0.25 band cannot detect the regressions the suite exists to catch. Reranking comes last; the anchor already handles the retrieval that matters.

That ordering is a claim about developer experience. Someone staring at 47 violations hits the generic fallback long before they hit the limits of technique retrieval.

Known limits

- Four context recipes plus a labelled fallback. The other ~40 axe rules get generic context.
- One performance issue type (LCP).
- Conversation is trimmed by dropping oldest turns, not summarised.
- `validate_fix` executes model-authored HTML in a headless throwaway browser. Fine locally, but it would need a hardened sandbox before multi-tenant use.